

HW2 — System Prompt Design

PantryPilot: A Waste-Minimizing Meal Planner

INFO7375 · Building AI Applications · Assignment 2

1. Guiding-Questions Worksheet

Section	Questions	Team Response
AI Role & Purpose	- What role is the AI playing? - What is its primary function for the user?	Role: PantryPilot, a waste-minimizing meal-planning orchestrator. Function: Turn the messy contents of a user's fridge/pantry into a multi-day meal plan and a minimal shopping list, prioritizing soon-to-spoil items and never breaking dietary constraints.
Full System Prompt	- What is your complete system prompt? - Role, instructions, constraints, output format?	Not one monolithic prompt: a global orchestrator prompt plus five decomposed stage prompts and one deterministic step. Each contains role, instructions, constraints, and a JSON output schema. Full text in Section 2.
Prompt Anatomy	- How is your prompt structured? - Does it separate context, task, constraints, output?	Every stage is split into ROLE · CONTEXT (injected variables) · TASK · CONSTRAINTS · OUTPUT SCHEMA · REFUSAL. Context is passed in as variables, never hardcoded.
Prompting Techniques	- Which techniques (zero/few-shot, decomposition)? - Why appropriate?	- Few-shot structural constraint — vision ingestion. - Chain-of-thought — freshness reasoning and ingredient-reuse planning. - Constrained generation — strict JSON schema at every stage. - Retrieval-grounded selection with pre-filtering — substitutions. - Decomposition + deterministic code — gap-list math done outside the model. Why: structured and safety-critical steps need determinism and grounding; free generation is reserved for recipe wording only.

Section	Questions	Team Response
Reasoning & Decomposition	- Does the prompt guide multiple steps? - How are complex tasks broken down?	Yes — a six-stage pipeline (vision → freshness → critical-priority → plan → substitution → gap list). Each stage does one job and passes validated JSON forward. CoT is used inside the freshness and planning stages.
Workflow-Oriented Behavior	- Multi-step / structured workflows? - Intermediate steps or reasoning patterns?	An orchestrator tracks the user's position in the journey via a local state object (session.json), updates the profile, and routes each message to the correct stage. Intermediate outputs are validated between stages.
Generated Knowledge / Self-Critique	- Intermediate outputs or validation? - How does this improve reliability?	Yes. Intermediate artifacts (detected inventory, freshness scores, critical list) plus validators — JSON-schema check, banned-ingredient scan, and set-math check — run before the user-facing answer, catching errors before they propagate down the pipeline.
Input Handling	- What inputs are expected? - How is incomplete/ambiguous input handled?	Inputs: a fridge photo, typed inventory, profile constraints, and intent messages. Vision output must be user-confirmed before use. Missing fields (servings, nights, time) trigger one clarifying question or a stated default; unparseable input is re-requested.
Output Structure	- What format do outputs follow? - Consistent across use cases?	Validated JSON contracts between stages; user-facing markdown for the plan and shopping list. The schema is fixed per stage, so output is consistent across all five use cases.
Constraints & Guardrails	- What should the model avoid? - How do you control hallucination and scope?	- Hard fail: never output anything on the user's restriction/allergen or dislikes list. - Ground, don't invent: freshness from a shelf-life table, substitutes from an approved diet-tagged table, recipes from retrieved candidates only. - Compute set math in code, not in text. - Refuse with a fixed string when no grounded, compliant answer exists. - Never infer visual staleness or exact expiry the model cannot know.

Section	Questions	Team Response
Prompt Sensitivity Testing	• Response to different phrasing? • What variations did you test?	Synonyms, list reordering, typos, added items, and reworded constraints/quantities, tested per stage. Full matrix in Section 4.
Stability & Robustness	• Consistent outputs across variations? • Where does it break down?	Temperature 0 on structured/safety stages; repeated-run consistency measured. Known break points: ingredients absent from the shelf-life table (default score 3 + flag) and low-quality photos (caught by the confirmation gate).
Sample Input 1 / Expected Output 1	• Realistic user input • Ideal response	Happy-path messy inventory → 3-night plan. See Section 3, Example 1.
Sample Input 2 / Expected Output 2	• Variation of the same input • How consistent is the response?	Reworded/synonym variant of Example 1 — should yield structurally equivalent output. See Section 3, Example 2.
Sample Input 3 (Edge Case) / Expected Output 3	• Incomplete or ambiguous input • How should the system handle it?	Constraint conflict (meat + strict-vegetarian) plus missing fields, and a no-valid-substitute case — handled by conflict routing, one clarifying question, and the fixed refusal string. See Section 3, Example 3.
Prompt Iteration & Fine-Tuning	• What issues did you observe? • What changes did you make and why?	Six documented fixes from v1 → v2 (unparseable prose, hardcoded recipe, meat/vegetarian contradiction, image-based freshness, unsafe substitution, model-side math). Full log in Section 5.
Alignment to Use Cases	• Which use cases are fully supported? • Any not well handled?	UC1 plan, UC2 substitution, UC3 gap list, UC4 constraint honoring, and UC5 use-it-up are all supported. Vision-based freshness is partial (mitigated by the shelf-life table + confirmation). Cross-store cheapest-price comparison is intentionally out of scope (see HW1 data-sourcing note).

2. Full System Prompt (Decomposed Pipeline)

PantryPilot does not use a single monolithic prompt. A global orchestrator sets shared rules; five stage prompts each do one job and pass validated JSON forward; one step is pure code. This limits context dilution, reduces tokens, and keeps calculation out of the model where it is prone to hallucinate.

2.0 Global Orchestrator Prompt

ROLE: You are PantryPilot, a waste-minimizing meal-planning assistant.
GOAL: From the user's available ingredients, produce a multi-day meal plan and a minimal shopping list that (a) uses soon-to-spoil items first, (b) never violates the user's hard dietary restrictions, and (c) fits their time and serving limits.

GLOBAL RULES (apply to every stage):

1. SAFETY IS A HARD CONSTRAINT. Never output a plan, recipe, or substitute containing an item on dietary restrictions or dislikes. One violation = failure.
2. GROUND, DON'T INVENT. Freshness scores come only from the provided shelf-life reference; substitutes only from the approved-substitutions reference; recipes only from the provided recipe candidates.
3. COMPUTE IN CODE, NOT IN TEXT. Do not do set arithmetic (e.g. shopping-list subtraction); that is handled deterministically outside the model.
4. CONTRACT + REFUSE. Return valid JSON matching each stage's schema. If you cannot produce a grounded, compliant answer, return the refusal object.
5. KNOW YOUR LIMITS. Do not infer an item's age or exact expiry from an image or its name; use only the perishability class of the ingredient TYPE.
6. Reserve creativity for recipe wording; all structured fields are deterministic.

2.1 Stage 1 — Refrigerator Vision Ingestion

Technique: few-shot structural constraints. Output is confirmed by the user before any downstream stage runs.

ROLE: Food-item extraction module.
CONTEXT: {{image}} -- a user photo of a fridge/freezer/pantry.
TASK: List only food items identifiable with reasonable confidence.
CONSTRAINTS: food items only; ignore brands, containers, shelving, non-food; use lowercase generic names.
OUTPUT (JSON only, no prose):
{
 "detected_items": [{
 "item": "<name>",
 "confidence": "high|medium|low"
 }],
 "needs_user_confirmation": true
}
FEW-SHOT EXAMPLE
Input: [photo: wilted greens, a carton, a rice jar, lemons, packaged raw meat]
Output: {
 "detected_items": [{
 "item": "spinach",
 "confidence": "high"
 }, {
 "item": "buttermilk",
 "confidence": "medium"
 }, {
 "item": "rice",
 "confidence": "high"
 }, {
 "item": "lemons",
 "confidence": "high"
 }, {
 "item": "chicken",
 "confidence": "medium"
 }],
 "needs_user_confirmation": true
}
NOTE: Downstream stages must NOT run until the user confirms or edits this list.

2.2 Stage 2 — Freshness / Perishability Scoring

Technique: chain-of-thought + retrieval grounding. Scores the ingredient TYPE from a shelf-life table, never the photo.

ROLE: Perishability classifier.
CONTEXT: confirmed items = {{confirmed items}}
shelf life reference = {{shelf life kb}}
TASK: For each item, look up its perishability CLASS in shelf_life_reference and assign a score:
5 = highly perishable / food-safety critical (use immediately)
4 = perishable (use soon)
3 = moderate
2 = long shelf life
1 = pantry stable
For each item, first write a one-line reason citing the reference class, then the score. Do NOT judge the physical condition of the specific item -- score the ingredient TYPE only. If an item is absent from the reference, assign 3 and set "in_reference": false.
OUTPUT (JSON, sorted by score descending):
{
 "ranked": [{
 "item": "spinach",
 "score": 5,
 "class": "leafy green",
 "reason": "leafy greens are highly perishable",
 "in_reference": true
 }]
}

2.3 Stage 3 — Critical-Priority Identification

Deterministic filter (score ≥ 4); the model only writes a short rationale.

THIS STAGE IS DETERMINISTIC (code, not generation):
critical_priority = [item for item in ranked if item.score >= 4]

The LLM is used ONLY to write a one-sentence rationale per item, grounded in the stage-2 reason (e.g. citing general food-safety guidance).
 OUTPUT (JSON):
 {"critical priority":["spinach","chicken"],
 "explanation":"Both are highly perishable / food-safety critical and should be used first to avoid waste and illness."}

2.4 Stage 4 — Constraint-Based Meal Planning

Technique: constrained generation with a vegetarian-flexible conflict-routing rule; recipes drawn only from retrieved candidates.

ROLE: Meal planner.
 CONTEXT:
 critical_priority = {{critical_priority}}
 full_inventory = {{inventory}}
 recipe_candidates = {{cleaned_api_recipes}} // retrieved; do NOT invent recipes
 profile = {{user_profile}}
 // diet_style: "vegetarian-flexible" | "strict-vegetarian" | "none"
 // dietary_restrictions (HARD), dislikes, time_limit_min, servings, nights
 TASK:
 1. Plan {{nights}} meals, each <= {{time limit min}} min, serving {{servings}}, drawn ONLY from recipe candidates.
 2. Maximize reuse: schedule soonest-to-spoil (critical priority) items earliest; reuse leftover components across nights.
 3. CONFLICT ROUTING for any inventory item that conflicts with diet_style:
 - strict-vegetarian: do NOT use it; add to "flagged for other use" with note "reserved for a non-vegetarian household member"; use a vegetarian protein from inventory/candidates instead.
 - vegetarian-flexible: it MAY be used, OR offer a vegetarian swap; state which.
 4. NEVER use any item in dietary_restrictions or dislikes (HARD).
 OUTPUT (JSON):
 {"plan":[{"night":1,"recipe":"...", "time min":25,"servings":2,
 "uses critical":["spinach"],"ingredients from inventory":["rice","lemons"],
 "extra_pantry_items":["salt","cumin"],"steps":["..."]},
 "flagged_for_other_use":[{"item":"chicken","reason":"..."}]}

2.5 Stage 5 — Substitution Rescue (Dynamic)

Technique: variable injection + retrieve-then-filter-then-select + fixed refusal. Dietary safety is enforced before the model chooses.

ROLE: Real-time substitution assistant.
 CONTEXT:
 recipe_context = {{recipe_context}}
 missing_ingredient = {{missing_ingredient}}
 dietary_restrictions = {{dietary_restrictions}} // HARD
 dislikes = {{dislikes}}
 approved_substitutions = {{substitution_kb}} // the ONLY allowed source
 PROCEDURE (in order):
 1. Retrieve candidate substitutes for missing ingredient from approved substitutions.
 2. REMOVE every candidate containing anything in dietary restrictions or dislikes (filter BEFORE choosing).
 3. From the remaining compliant candidates, pick the simplest, most common one.
 4. Give how to prepare it and how to integrate it into recipe context.
 5. If NO compliant candidate remains, output the refusal object verbatim -- do NOT invent a substitute.
 OUTPUT (JSON):
 success: {"status":"ok","substitute":"almond milk + lemon juice",
 "prep":"mix 1 cup almond milk with 1 tbsp lemon juice; rest 5 min",
 "integration":"use 1:1 in place of buttermilk","source":"approved substitutions"}
 refusal: {"status":"no substitute",
 "message":"No suitable substitute found. Modify the recipe accordingly."}

2.6 Gap-Only Shopping List

Deterministic set math in code; the model only formats the result.

DETERMINISTIC CODE (no LLM):
 gaps = required_ingredients(plan) - current_inventory - pantry_staples
 LLM (FORMATTING ONLY):
 Given {{gaps}}, group items by grocery-store aisle and render as markdown.
 Do NOT add, remove, rename, or recompute any item.

3. Example Inputs & Outputs

Example 1 — Messy inventory → plan (happy path, UC1)

Input: “Chicken thighs, half a bag of spinach going soft, rice, some lemons — vegetarian-flexible, 30 min max, 2 people, 3 nights.”

Expected behavior:

- Freshness (from shelf-life table): spinach 5, chicken 5, lemons 2, rice 1.
- Critical priority: spinach, chicken.
- diet_style = vegetarian-flexible → plan uses spinach across nights; for chicken it either uses it or offers a paneer/tofu swap, stating which.
- Three meals, each ≤ 30 min / 2 servings, reusing spinach + rice + lemon across nights.
- Gap list = required – inventory – pantry staples (computed in code), grouped by aisle.

Expected output (abbreviated):

```
{ "plan": [
  { "night": 1, "recipe": "Spinach & Lemon Rice", "time min": 20, "servings": 2,
    "uses critical": ["spinach"], "ingredients from inventory": ["spinach", "rice", "lemons"],
    "extra pantry items": ["salt", "cumin"] },
  { "night": 2, "recipe": "Paneer-Spinach Skillet (veg swap for chicken)", "time min": 25,
    "servings": 2, "uses critical": ["spinach"], "extra pantry items": ["paneer"] },
  { "night": 3, "recipe": "Lemon Rice Bowl", "time min": 15, "servings": 2,
    "ingredients from inventory": ["rice", "lemons"] },
  "flagged for other use": [{"item": "chicken",
    "reason": "vegetarian swap chosen; chicken offered as optional add-on"}] }
```

Example 2 — Reworded variant (stability, UC1)

Input: “I’ve got some baby spinach that’s wilting, a couple lemons, leftover cooked rice, and 2 chicken thighs. I sometimes eat meat. Two of us, three dinners, nothing over half an hour.”

Expected behavior:

- Same critical items (spinach, chicken) and the same vegetarian-flexible routing as Example 1.
- Freshness scores within ± 1 of Example 1; “baby spinach” maps to the same leafy-green class.
- Plan is structurally equivalent (3 meals, ≤ 30 min, 2 servings, spinach reused first).

Consistency check: Because scoring is grounded in the shelf-life class and the schema is fixed, the reworded input produces the same structured decisions — this is the evidence of stability.

Example 3 — Edge case: constraint conflict + missing fields

Input: “idk, some stuff — chicken and paneer. I’m strict vegetarian. make dinner.”

Two problems: (a) chicken conflicts with strict-vegetarian; (b) servings, nights, and time are missing.

Expected behavior:

- Ask exactly ONE clarifying question for the missing fields, or apply stated defaults (e.g. 2 servings, 1 night, 30 min) and say so.
- Conflict routing: chicken → flagged_for_other_use, note “reserved for a non-vegetarian household member”; the plan is built around paneer only.
- Chicken NEVER appears in a vegetarian plan — the contradiction is resolved by routing, not by silently using or dropping it.

- If a chosen recipe needs an ingredient with no compliant substitute, return the fixed refusal string rather than inventing one.

Expected output (abbreviated):

```
{
  "clarification": "Planning 1 dinner for 2, 30-min max (defaults) -- reply to change.",
  "plan": [
    {
      "night": 1,
      "recipe": "Paneer Bhurji",
      "time_min": 20,
      "servings": 2,
      "ingredients_from_inventory": ["paneer"],
      "extra_pantry_items": ["onion", "spices"]
    },
    {
      "flagged_for_other_use": [
        {
          "item": "chicken",
          "reason": "reserved for a non-vegetarian household member"
        }
      ]
    }
  ]
}
```

4. Prompt Sensitivity & Stability

Method: for each stage, run ~5 base inputs and 3–4 perturbed variants of each; for JSON stages check schema validity and score drift, for the planner check that every constraint still holds. Stability: run the same input 5 times at temperature 0 and measure identical-output / score-match rate.

Perturbation	Example	What we check / target
Synonym swap	spinach → “baby spinach”	Maps to the same leafy-green class; freshness score stays within ± 1 .
List reordering	rice, spinach, lemons → lemons, rice, spinach	Ranking and plan are identical (order-independent).
Typo	“chiken”, “spinch”	Still detected/normalized; if unsure, item is flagged for confirmation rather than guessed.
Added item	extra onion appears	Plan may include it; all hard constraints still hold.
Reworded constraint	“no dairy” vs “dairy-free”	Same dietary filtering applied before any selection.
Quantity reword	“2 thighs” vs “a couple chicken thighs”	No schema break; structured fields unchanged.

Evaluation harness (how outputs are judged)

- **Deterministic validators (no LLM):** JSON-schema validation; a banned-ingredient scan (constraint-violation rate, target = 0); a set-math check on the gap list.
- **LLM-as-a-judge:** scores faithfulness, constraint adherence (binary — any dietary violation = automatic fail), clarity, and beginner-friendliness, 1–5 each.
- **Metrics reported:** constraint-violation rate (0), JSON-valid %, substitution-grounding %, and score-consistency %.

Known break points

- Ingredient absent from the shelf-life table → default score 3 and flag for confirmation.
- Low-quality or cluttered photo → caught by the Stage-1 confirmation gate before it propagates.
- Conflicting constraints → resolved by the Stage-4 routing rule, not left undefined.

5. Prompt Iteration & Fine-Tuning Log

Each entry is a real correction made after testing, framed as symptom → change → result.

Ver.	Issue (symptom)	Change	Result
v1→v2	Freshness emitted as prose sentences	Enforced a strict JSON schema with numeric scores	Backend parses 100% of outputs; scores usable in code
v1→v2	Substitution prompt hardcoded “Lemon Spinach Bowls”	Replaced with variable injection {{recipe_context}}, {{missing_ingredient}}	Generalizes to any recipe/ingredient
v1→v2	Chicken passed into a strict-[vegetarian] plan	Added diet_style conflict-routing rule	Meat flagged for other use; plan stays compliant; thesis preserved
v1→v2	Freshness “estimated from the image”	Score the ingredient TYPE from a shelf-life table + add a confirmation gate	Deterministic and honest; no visual-staleness hallucination
v1→v2	Open-ended substitution could suggest dairy to a dairy-free user	Retrieve from an approved diet-tagged table + filter BEFORE selecting + fixed refusal	Non-compliant substitutes are structurally impossible
v1→v2	Model did the gap-list subtraction (arithmetic errors)	Moved set math to Python; LLM formats only	Exact gap lists; no fabricated/dropped items
v1→v2	Vision errors propagated through the whole pipeline	Inserted a human-confirmation step after Stage 1	Stops error cascades at the source

Sources & Tools

- AI tools: structural critique of the draft prompts was reviewed with Google Gemini; the fixes above were selected and re-designed by the team.
- Data: shelf-life / perishability classes derived from public food-safety storage guidance; substitution table curated and diet-tagged by the team; recipe candidates from the Spoonacular API (see HW1 references).